

机器视觉实验报告

专业 智能科学与技术 姓名 史峰源 学号 2412526

1 实验目的

1. 掌握直方图均衡化的数学原理与实现流程，能够使用 C++ 自行编写底层代码实现图像的对比度增强。
2. 深入理解空间域滤波器的基本概念，掌握平滑滤波器与差分滤波器在图像处理中的先后逻辑关系。
3. 掌握使用一阶差分滤波器（如 Sobel 算子）进行图像边缘检测的原理与 C++ 编程实现方法。
4. 熟悉 OpenCV 库的基本数据结构（如 Mat）和基础图像读写、灰度转换与显示操作。

2 实验原理

2.1 直方图均衡化

直方图均衡化是一种通过改变图像的直方图分布，使其变为近似均匀分布，从而增强图像全局对比度的方法。对于灰度级为 k （通常为 256），大小为 $M \times N$ 的图像，其算法原理如下：

1. 统计原图像的直方图分布 $H[p]$ ，其中 $p \in [0, k-1]$ 代表灰度级。
2. 计算累计直方图 H_c ，其定义为： $H_c[p] = H_c[p-1] + H[p]$ （且 $H_c[0] = H[0]$ ）。
3. 根据累计直方图构建映射查找表 $T[p]$ 。为了将灰度级重新映射到 $[0, k-1]$ 范围内，变换公式为：

$$T[p] = \text{round} \left(\frac{k-1}{MN} H_c[p] \right)$$

4. 遍历原图像，将原图像中的像素值 p 替换为查找表中的对应值 $T[p]$ ，获得均衡化后的图像。

2.2 使用差分滤波器的边缘检测

边缘是图像中像素灰度值发生剧烈变化的位置，在数学上对应于图像函数一阶导数的极值点或二阶导数的过零点。

1. **平滑处理**：由于导数（微分）操作对噪声非常敏感，在进行边缘检测前，必须先使用平滑滤波器（如高斯滤波器）处理原始图像，以降低离散噪声对梯度的影响。
2. **差分运算（以 Sobel 算子为例）**：Sobel 算子是一种带有方向性的有限窗口一阶差分滤波器。它包含两个 3×3 的卷积核，分别用于提取水平方向（y 方向）和垂直方向（x 方向）的边缘梯度：

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * I \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * I$$

3. **梯度幅值**：计算出水平和垂直梯度后，通常通过计算绝对值之和 $|G| = |G_x| + |G_y|$ 来近似总的梯度幅值，以此作为最终的边缘图像。

3 实验步骤

1. 配置 C++ 开发环境，并引入 OpenCV 库。在工程相对路径下准备一张测试图像 `testing.jpg`。

2. 直方图均衡化实验步骤：

- 读取彩色测试图并转换为单通道灰度图。
- 编写 `myEqualizeHist` 函数，创建长度为 256 的数组统计直方图和累计直方图。
- 根据公式计算映射关系并生成查找表（LUT）。
- 根据查找表遍历灰度图像素进行赋值，并调用 `imshow` 对比显示原图与均衡化效果。

3. 边缘检测实验步骤：

- 读取图片并转为灰度图。
- 编写 `myEdgeDetect` 函数。首先使用 `GaussianBlur` 进行 3×3 的高斯平滑滤波去噪。
- 调用 `Sobel` 函数分别求取 X 方向和 Y 方向的一阶差分（注意使用 16 位有符号整型 `CV_16S` 以防截断）。
- 对梯度结果取绝对值并转换回 8 位无符号整型（`CV_8U`），使用加权相加合成最终的边缘检测图并显示。

4 程序代码

注：根据实验要求，代码中已统一使用相对路径 `"testing.jpg"` 读取图片。

4.1 直方图均衡化 (C++)

```
#include <iostream>
#include <fstream>
#include "opencv2/opencv.hpp"
#include "opencv2/imgproc/imgproc.hpp"
#include "opencv2/highgui/highgui.hpp"
#include <stdio.h>
#include <cmath>
using namespace cv;
using namespace std;

Mat myEqualizeHist(Mat img)
{
    //确保输入单通道灰度图片
    CV_Assert(img.type()==CV_8UC1);
    Mat EqualizedImg;
    /*
    完善直方图均衡化的计算过程

    */
    EqualizedImg.create(img.rows,img.cols,img.type());
    int k=256;
```

```

int H[256]={0};
int M=img.rows;
int N=img.cols;
int MN=M*N;

//形成直方图
for(int i=0;i<M;i++){
    for(int j=0;j<N;j++){
        int pixelValue=img.at<uchar>(i,j);
        H[pixelValue]++;
    }
}

//形成累计直方图
int Hc[256]={0};
Hc[0]=H[0];
for(int p=1;p<k;p++){
    Hc[p]=Hc[p-1]+H[p];
}

//设置查找表
uchar T[256]={0};
double factor=(double)(k-1)/MN;
for(int p=0;p<k;p++){
    T[p]=(uchar)cvRound(factor*Hc[p]);
}

for(int i=0;i<M;i++){
    for(int j=0;j<N;j++){
        int originalPixel=img.at<char>(i,j);
        EqualizedImg.at<char>(i,j)=T[originalPixel];
    }
}

//返回原图像经过直方图均衡化后的变换结果
return EqualizedImg;
}

void main()
{
    Mat input = imread("testing.jpg");

    Mat gray;
    //彩色图转为灰度图
    cvtColor(input, gray, COLOR_BGR2GRAY);

    //直方图均衡化，需编程实现
    Mat EqualizedImg = myEqualizeHist(gray);

    imshow("input", input);
}

```

```

imshow("EqualizedImg", EqualizedImg);
waitKey(0);
}

```

4.2 使用差分滤波器的边缘检测 (C++)

```

#include <iostream>
#include <fstream>
#include "opencv2/opencv.hpp"
#include "opencv2/imgproc/imgproc.hpp"
#include "opencv2/highgui/highgui.hpp"
#include <cmath>
#include <stdio.h>

using namespace cv;
using namespace std;

Mat myEdgeDetect(Mat img)
{
    Mat EdgeImg;
    CV_Assert(img.type()==CV_8UC1);

    //先进性平滑滤波，再使用一阶差分滤波器

    Mat blurred;
    GaussianBlur(img,blurred,Size(3,3),0,0);

    Mat grad_x,grad_y;
    Mat abs_grad_x,abs_grad_y;

    Sobel(blurred,grad_x,CV_16S,1,0,3);
    Sobel(blurred,grad_y,CV_16S,0,1,3);

    convertScaleAbs(grad_x,abs_grad_x);
    convertScaleAbs(grad_y,abs_grad_y);

    addWeighted(abs_grad_x,0.5,abs_grad_y,0.5,0,EdgeImg);
    /*
    完善使用差分滤波器的边缘检测计算过程（利用线性滤波）

    */

    //返回原图像经过边缘检测的结果EdgeImg
    return EdgeImg;
}

```

```

int main()
{
    Mat input = imread("testing.jpg");

    Mat gray;
    //彩色图转为灰度图
    cvtColor(input, gray, COLOR_BGR2GRAY);

    //使用差分滤波器的边缘检测（利用线性滤波），需编程实现
    Mat EdgeImg = myEdgeDetect(gray);

    imshow("input", input);
    imshow("EdgeImg", EdgeImg);
    waitKey(0);
    return 0;
}

```

5 实验结果显示

5.1 直方图均衡化结果

（注：此处为实验截图展示，为经过代码处理后的均衡化图像。）



图 1: = 实验结果图像

5.2 边缘检测结果

（注：此处为实验截图展示。为经过高斯平滑加 *Sobel* 算子处理后的边缘检测图像。）



图 2: 基于差分滤波器的边缘检测结果对比

6 实验分析总结

通过本次实验，我成功完成了直方图均衡化和基于差分滤波器的边缘检测两个经典机器视觉算法的底层代码实现：

1. 直方图均衡化分析：实验结果表明，经过算法处理后，原图中过于集中、对比度较低的灰度区域被有效拉伸。像素值在 0 255 的范围内分布更加均匀。在视觉感官上，图像原本昏暗或过曝的细节变得清晰可见。通过手动编写累积分布函数（CDF）的映射代码，我深刻理解了图像点运算的本质机制。

2. 边缘检测分析：在使用差分滤波器前引入了高斯平滑处理。如果略去高斯平滑步骤，图像中的孤立噪点会被差分滤波器放大，导致边缘图中出现大量杂散的白点（伪边缘）。通过先后组合“平滑滤波器（低通）”和“差分滤波器（高通，如 Sobel）”，程序不仅成功提取了图像中的物体轮廓结构，同时较好地抑制了背景噪声。

总结：本次实验不仅锻炼了利用 C++ 处理图像矩阵底层数据的能力，还加深了对《机器视觉》课程中像素点操作（空间域变换）和局部模板卷积运算的理解，圆满达到了预期的实验目的。